

Lec.5

Unsupervised learning: A means of modifying the weights of a neural net without specifying the desired output for any input patterns. Used in self-organizing neural nets for clustering data, extracting principal components, or curve fitting.

Supervised training: Process of adjusting the weights in a neural net using a learning algorithm; the desired output for each of a set of training input vectors is presented to the net. Many iterations through the training data may be required.

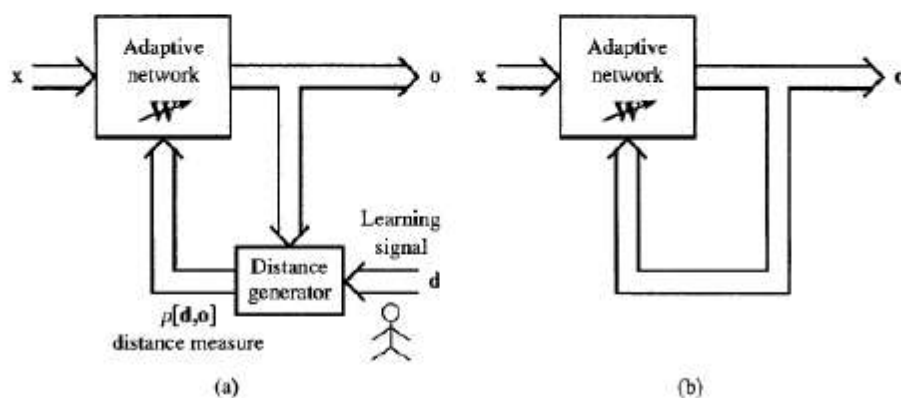


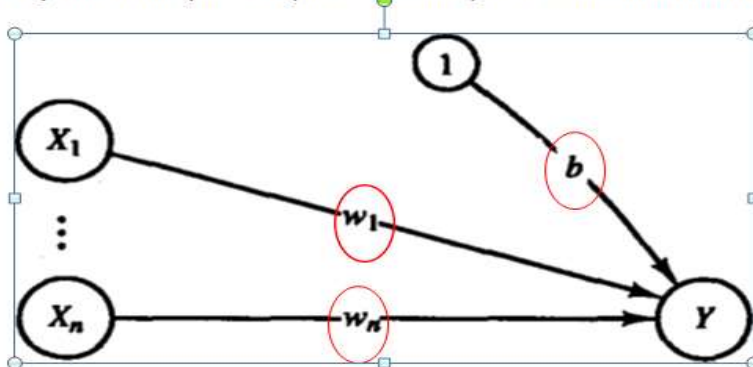
Figure 2.19 Block diagram for explanation of basic learning modes: (a) supervised learning and (b) unsupervised learning.

Adaline

- 1- Architecture
- 2- Algorithm
- 3- Application

ARCHITECTURE

An Adaline is a single unit (neuron) that receives input from several units. It also receives input from a "unit" whose signal is always + 1, in order for the bias weight to be trained by the same process (the delta rule) as is used to train the other weights.



2- Algorithms

- Step 0. Initialize weights.
(Small random values are usually used.)
Set learning rate α .
(See comments following algorithm.)
- Step 1. While stopping condition is false, do Steps 2–6.
- Step 2. For each bipolar training pair $s; t$, do Steps 3–5.
- Step 3. Set activations of input units, $i = 1, \dots, n$:
- $$x_i = s_i.$$
- Step 4. Compute net input to output unit:
- $$y_{in} = b + \sum_i x_i w_i.$$
- Step 5. Update bias and weights, $i = 1, \dots, n$:
- $$b(\text{new}) = b(\text{old}) + \alpha(t - y_{in}).$$
- $$w_i(\text{new}) = w_i(\text{old}) + \alpha(t - y_{in})x_i.$$
- Step 6. Test for stopping condition:
If the largest weight change that occurred in Step 2 is smaller than a specified tolerance, then stop; otherwise continue.

$$\Delta W1 = x1 * T$$

$$\Delta W2 = x2 * T$$

$$\Delta B = b * T$$

$$E = (T - y_{in})^2$$

3-Application

APPLICATIONS

After training, an Adaline unit can be used to classify input patterns. If the target values are bivalent (binary or bipolar), a step function can be applied as the activation function for the output unit. The following procedure shows the step function for bipolar targets, the most common case:

- Step 0.** Initialize weights
(from Adaline training algorithm given in Section 2.4.2).
- Step 1.** For each bipolar input vector $\mathbf{x}$, do Steps 2–4.
- Step 2.** Set activations of the input units to $\mathbf{x}$.
- Step 3.** Compute net input to output unit:
$$y_{in} = b + \sum_i x_i w_i.$$
- Step 4.** Apply the activation function:
$$y = \begin{cases} 1 & \text{if } y_{in} \geq 0; \\ -1 & \text{if } y_{in} < 0. \end{cases}$$

Q1: An Adaline for the OR_GATE bipolar input and target with the initial weight (0.1) and the learning rate (0.1)

X1	X2	B	T	YIN	T-YIN	$\Delta W1$ $\alpha(t-yin)x1$	$\Delta W2$ $\alpha(t-yin)x2$	ΔB $\alpha(t-yin)$	W1 (0.1)	W2 (0.1)	B (0.1)	ERROR $E=(T-yin)^2$
1	1	1	1	0.3	0.7	0.07	0.07	0.07	0.17	0.17	0.17	0.49
1	-1	1	1	0.17	0.83	0.083	-0.083	0.083	0.253	0.087	0.253	0.6889
-1	1	1	1	0.087	0.913	-0.0913	0.0913	0.0913	0.1617	0.1783	0.3443	0.83356
-1	-1	1	-1	0.0043	-1.0043	0.10043	0.10043	-0.10043	0.26213	0.2787	0.2439	1.00861849 E= 3.02107849

Epoch 2

X1	X2	B	T	YIN	T-YIN	$\Delta W1$	$\Delta W2$	ΔB	W1 (0.26213)	W2 (0.2787)	B (0.2439)	ERROR
1	1	1	1	0.78473	0.21557	0.021557	0.021557	0.021557	0.28368	0.3003	0.2654	0.046470
1	-1	1	1	0.2488	0.7512	0.0751	-0.0751	0.0751	0.3588	0.2251	0.3405	0.5643014
-1	1	1	1	0.2069	0.7931	-0.0793	0.0793	0.0793	0.2795	0.3044	0.4198	0.62900761
-1	-1	1	-1	-0.1641	-0.8359	0.0836	0.0836	-0.0836	0.3631	0.388	0.3362	0.69872881 E=1.984977

Epoch 3

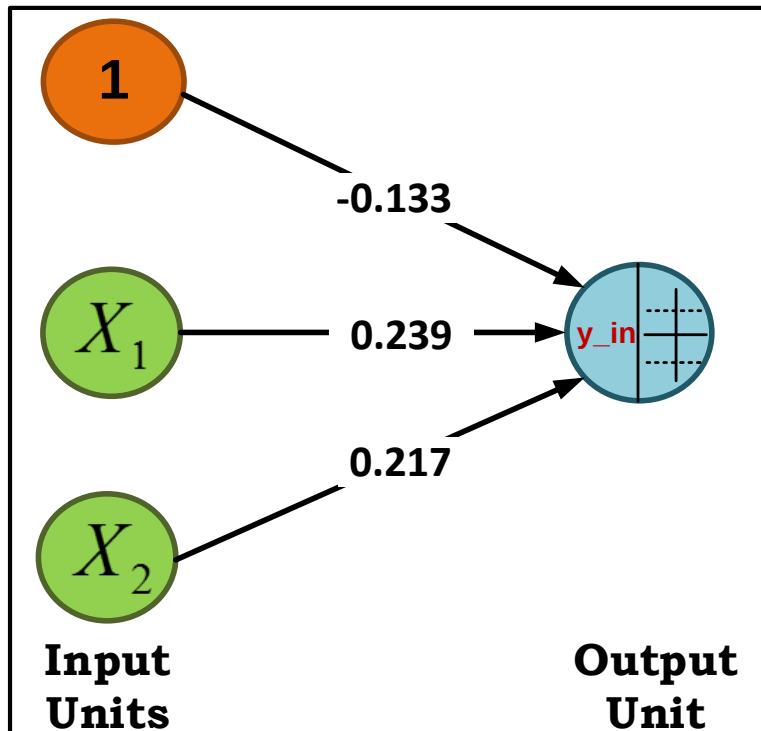
X1	X2	B	T	YIN	T-YIN	$\Delta W1$	$\Delta W2$	ΔB	W1 ()	W2 (0)	B)	ERROR
1	1	1	1	1.0873	0.0873	-0.0087	0.0087	-0.0087	0.3543	0.3793	0.327	0.00762129
1	-1	1	1	0.3025	0.6975	0.0697	-0.0697	0.0697	0.4241	0.3096	0.3973	0.48650625
-1	1	1	1	0.2827	0.7173	-0.0717	0.0717	0.0717	0.3523	0.3813	0.4690	0.51451929
-1	-1	1	-1	- 0.2647	-0.7353	0.0735	0.0735	-0.0735	0.4259	0.4548	0.3954	0.54125999 E=1.54990132

Epoch 4

X1	X2	B	T	YIN	T-YIN	$\Delta W1$	$\Delta W2$	ΔB	W1 ()	W2 ()	B	ERROR
1	1	1	1	1.2761	0.2761	-0.0276	-0.0276	-0.0276	0.17	0.3983	0.3678	0.07623121
1	-1	1	1	0.3389	0.6611	0.0661	-0.0661	0.0661	0.253	0.4644	0.4339	0.43705321
-1	1	1	1	0.3307	0.6693	-0.0669	0.0669	0.0669	0.1617	0.3974	0.5009	0.44796249
-1	-1	1	-1	- 0.3246	-0.6754	0.0675	0.0675	-0.0675	0.26213	0.4650	0.4333	0.45616516 E=1.41741207

Adaline

Epoch 1



Epoch 5

The weights at the end of Epoch 5 is approximately:

$$w1 = 0.5 \quad , \quad w2 = 0.4 \quad , \quad b = -0.3$$

Therefore, the least mean squares (LMS) error is calculated

Inputs			t	y_in	$E = (t - y_{in})^2$
x1	x2	b		w1=0.5, w2=0.4, b= -0.3	
1	1	1	1	0.6	$(1 - 0.6)^2 = 0.16$
1	-1	1	-1	-0.2	$(-1 + 0.2)^2 = 0.64$
-1	1	1	-1	-0.4	$(-1 + 0.4)^2 = 0.36$
-1	-1	1	-1	-1.2	$(-1 + 1.2)^2 = 0.04$

- **Homework**

1- ADALINE net for the ANDNOT function (bipolar inputs and targets)

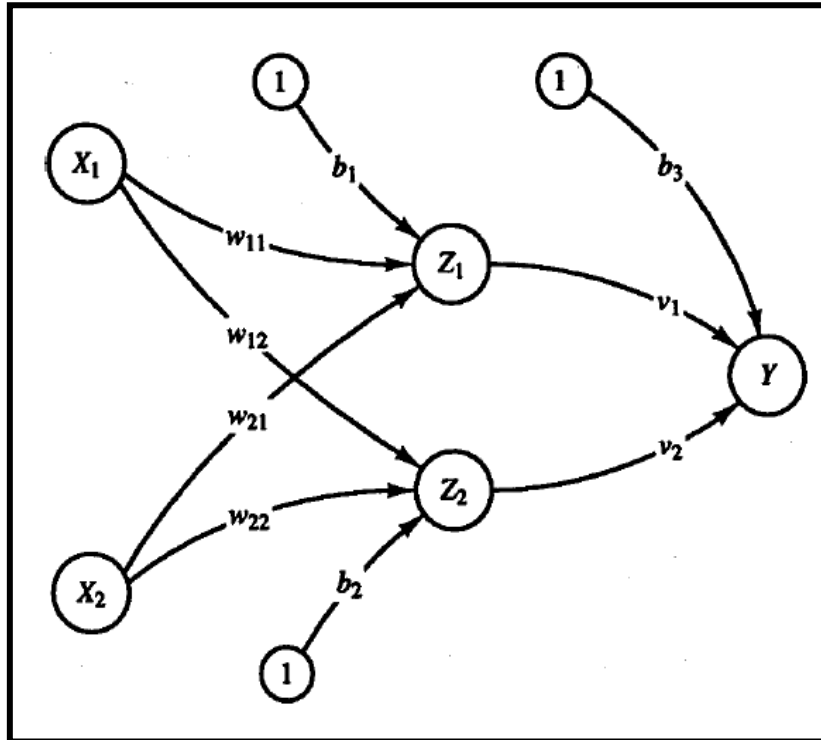
$w_1=w_2=b=0.2$, Learning rate=0.2 (6 Epoch)

2- ADALINE net for the OR function (bipolar inputs and targets)

$w_1=w_2=b=0.1$, Learning rate=0.1 ($E < 1.4$)

Madaline

MADALINE consists of Many Adaptive Linear Neurons (ADALINE) arranged in a multilayer net. A simple MADALINE net is illustrated in bellow Figure



The outputs of the two hidden ADALINES, Z_1 and Z_2 are determined by signals from the same input units X_1 and X_2

As with the Adalines discussed previously, each output signal is the result of applying a threshold function to the unit's net input. Thus, y is a nonlinear function of the input vector (x_1, x_2) .

The use of the hidden units Z_1 and Z_2 , give the net computational capabilities not found in single layer nets, but also complicate the training process.

Training Algorithm for Madaline (MRI). The activation function for units Z_1 , Z_2 , and Y is

$$f(x) = \begin{cases} 1 & \text{if } x \geq 0; \\ -1 & \text{if } x < 0. \end{cases}$$

Step 0. Initialize weights:

Weights v_1 and v_2 and the bias b_3 are set as described;

small random values are usually used for **Adaline** weights.

Set the learning rate a as in the **Adaline** training algorithm (a small value).

Step 1. While stopping condition is false, do Steps 2–8.

Step 2. For each bipolar training pair, $s:t$, do Steps 3–7.

Step 3. Set activations of input units:

$$x_i = s_i.$$

Step 4. Compute net input to each hidden **Adaline** unit:

$$z_{in1} = b_1 + x_1w_{11} + x_2w_{21},$$

$$z_{in2} = b_2 + x_1w_{12} + x_2w_{22}.$$

Step 5. Determine output of each hidden Adaline unit:

$$z_1 = f(z_{in_1}),$$

$$z_2 = f(z_{in_2}).$$

Step 6. Determine output of net:

$$y_{in} = b_3 + z_1v_1 + z_2v_2;$$

$$y = f(y_{in}).$$

Step 7. Determine error and update weights:

If $t = y$, no weight updates are performed.
Otherwise:

If $t = 1$, then update weights on Z_J ,
the unit whose net input is closest to 0,

$$b_J(\text{new}) = b_J(\text{old}) + \alpha(1 - z_{in_J}),$$

$$w_{iJ}(\text{new}) = w_{iJ}(\text{old}) + \alpha(1 - z_{in_J})x_i;$$

If $t = -1$, then update weights on all units Z_k that have positive net input,

$$b_k(\text{new}) = b_k(\text{old}) + \alpha(-1 - z_{in_k}),$$

$$w_{ik}(\text{new}) = w_{ik}(\text{old}) + \alpha(-1 - z_{in_k})x_i.$$

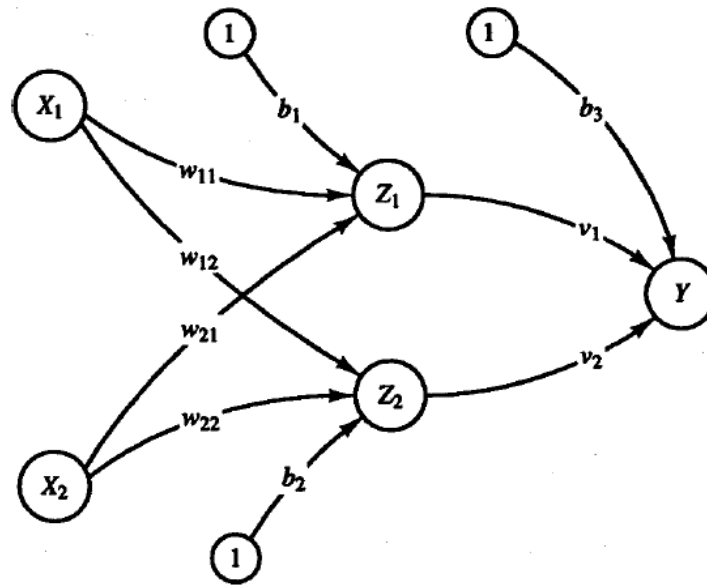
Step 8. Test stopping condition.

If weight changes have stopped (or reached an acceptable level), or if a specified maximum number of weight update iterations (Step 2) have been performed, then stop; otherwise continue.

Example 2.20: Training a MADALINE for the XOR function

This example illustrates the use of the MRI algorithm to train a MADALLNE to solve the XOR problem.

The training patterns are:



Initial values

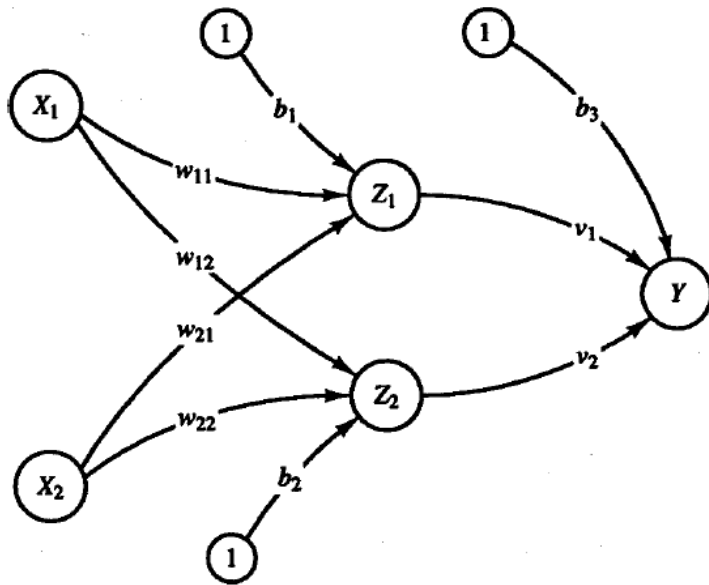
Learning rate $\alpha' = 0.5$

$W_{11} = 0.05, w_{21} = 0.2, b_1 = 0.3$

$W_{12} = 0.1, w_{22} = 0.2, b_2 = 0.5$

$V_1 = v_2 = b_3 = 0.5$

X1	X2	T
1	1	-1
1	-1	1
-1	1	1
-1	-1	-1



The training algorithm is followed.

For first epoch, first input pair (1,1) : 1.

Step 1: Weights and bias are initialized.

$$\begin{aligned}
 v_1 = v_2 = b_3 &= 0.5 \\
 w_{11} = 0.05, w_{21} &= 0.2, b_1 = 0.3 \\
 w_{12} = 0.1, w_{22} &= 0.2, b_2 = 0.15 \\
 \alpha &= 0.5
 \end{aligned}$$

Step 2: Begin training

Step 3: For input and training pair (1,1) : 1, perform steps 4 – 8.

Step 4: $x_i = S_i$

$$x = (1 \ 1)$$

Step 5: Calculate the net input

$$\begin{aligned}
 z_{in1} &= b_1 + x_1 w_{11} + x_2 w_{21} \\
 &= 0.3 + 1 \times 0.05 + 1 \times 0.2 \\
 &= 0.55 \\
 z_{in2} &= b_2 + x_1 w_{12} + x_2 w_{22} \\
 &= 0.15 + 1 \times 0.1 + 1 \times 0.2 \\
 &= 0.45
 \end{aligned}$$

Step 6: Find output z_1 and z_2 apply activations

$$\begin{aligned}
 z_1 &= f(z_{in1}) = 1 \\
 z_2 &= f(z_{in2}) = 1
 \end{aligned}$$

Step 7: Calculate net input of output

$$\begin{aligned}
 y_{in} &= b_3 + v_1 z_1 + v_2 z_2 \\
 &= 0.5 + 1 \times 0.5 + 1 \times 0.5 \\
 &= 1.5
 \end{aligned}$$

Find the output, and apply activations.

$$y = f(y_{in}) = 1$$

Inputs				t	z_{in1}	z_{in2}	z_1	z_2	y_{in}	y	w_{11}	w_{21}	b_1	w_{12}	w_{22}	b_2
	x_1	x_2	b													
Epoch 1	1	1	1	-1	0.55	0.45	1	1	1.5	1	-0.725	-0.575	-0.475	-0.625	-0.525	-0.575
	1	-1	1	1	-0.625	-0.675	-1	-1	-0.5	-1	0.0875	-1.3875	0.3375	-0.625	-0.525	-0.575
	-1	1	1	1	-1.1375	-0.475	-1	-1	-0.5	-1	0.0875	-1.3875	0.3375	-1.3625	0.2125	0.1625
	-1	-1	1	-1	1.6375	1.3125	1	1	1.5	1	1.4065	-0.0685	-0.9815	-0.207	1.369	-0.994

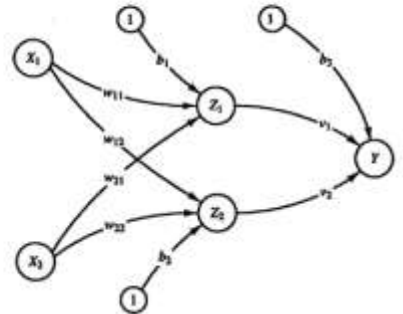
$$w_{11} = 0.05, w_{21} = 0.2, b_1 = 0.3$$

$$w_{12} = 0.1, w_{22} = 0.2, b_2 = 0.5$$

$$v_1 = v_2 = b_3 = 0.5$$

$$z_{in1} = b_1 + x_1 w_{11} + x_2 w_{21}$$

$$z_{in2} = b_2 + x_1 w_{12} + x_2 w_{22}$$



Inputs				t	z_{in1}	z_{in2}	z_1	z_2	y_{in}	y	w_{11}	w_{21}	b_1	w_{12}	w_{22}	b_2
	x_1	x_2	b													
Epoch 1	1	1	1	-1	0.55	0.45	1	1	1.5	1	-0.725	-0.575	-0.475	-0.625	-0.525	-0.575
	1	-1	1	1	-0.625	-0.675	-1	-1	-0.5	-1	0.0875	-1.3875	0.3375	-0.625	-0.525	-0.575
	-1	1	1	1	-1.1375	-0.475	-1	-1	-0.5	-1	0.0875	-1.3875	0.3375	-1.3625	0.2125	0.1625
	-1	-1	1	-1	1.6375	1.3125	1	1	1.5	1	1.4065	-0.0685	-0.9815	-0.207	1.369	-0.994

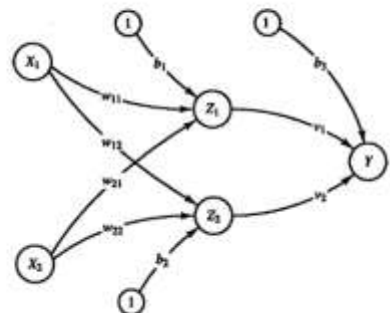
$$w_{11} = 0.05, w_{21} = 0.2, b_1 = 0.3$$

$$w_{12} = 0.1, w_{22} = 0.2, b_2 = 0.5$$

$$v_1 = v_2 = b_3 = 0.5$$

$$z_{in1} = b_1 + x_1 w_{11} + x_2 w_{21}$$

$$z_{in2} = b_2 + x_1 w_{12} + x_2 w_{22}$$



$$w_{11}(\text{new}) = w_{11}(\text{old}) + \alpha(-1 - z_{in1}) \times 1$$

$$= 0.05 + 0.5(-1 - 0.55) \times 1$$

$$= -0.725$$

$$w_{21}(\text{new}) = w_{21}(\text{old}) + \alpha(-1 - z_{in1}) \times 2$$

$$= 0.2 + 0.5(-1 - 0.55) \times 2$$

$$= -0.575$$

$$b_1(\text{new}) = b_1(\text{old}) + \alpha(-1 - z_{in1})$$

$$= 0.3 + 0.5(-1 - 0.55)$$

$$= -0.475$$

$$w_{12}(\text{new}) = w_{12}(\text{old}) + \alpha(-1 - z_{in2}) \times 1$$

$$= 0.1 + 0.5(-1 - 0.45) \times 1$$

$$= -0.625$$

$$w_{22}(\text{new}) = w_{22}(\text{old}) + \alpha(-1 - z_{in2}) \times 2$$

$$= 0.2 + 0.5(-1 - 0.45) \times 2$$

$$= -0.525$$

$$b_2(\text{new}) = b_2(\text{old}) + \alpha(-1 - z_{in2})$$

$$= 0.5 + 0.5(-1 - 0.45)$$

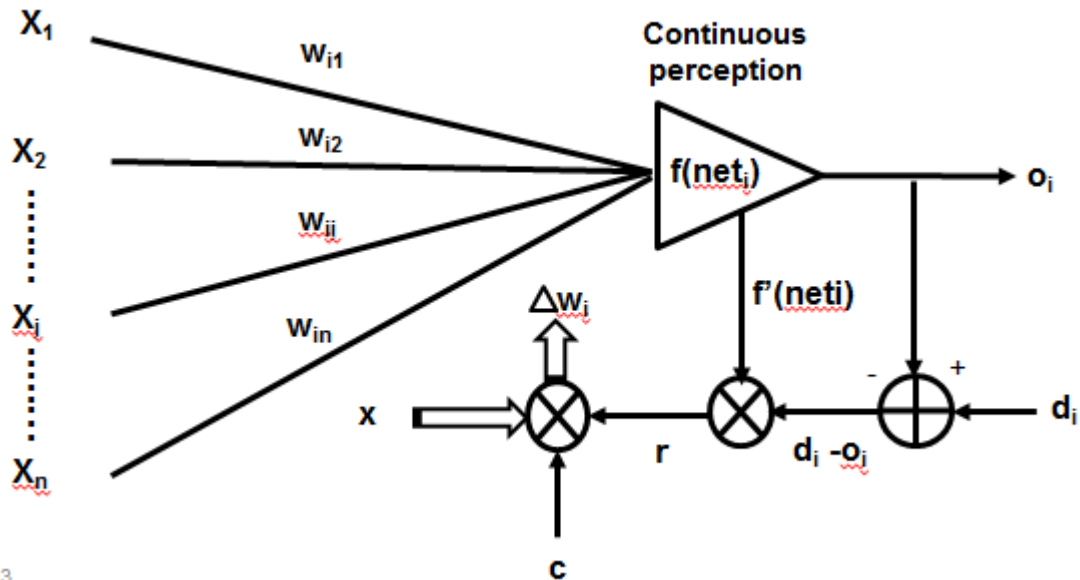
$$= -0.575$$

Inputs				t	z_{in1}	z_{in2}	z_1	z_2	y_{in}	y	w_{11}	w_{21}	b_1	w_{12}	w_{22}	b_2
	x_1	x_2	b													
Epoch 2	1	1	1	-1	0.3565	0.168	1	1	1.5	1	0.7285	-0.7465	-1.6595	-0.791	0.785	-1.578
	1	-1	1	1	-0.1845	-3.154	-1	-1	-0.5	-1	1.3205	-1.339	-1.068	-0.791	0.785	-1.578
	-1	1	1	1	-3.728	-0.002	-1	-1	-0.5	-1	1.3205	-1.339	-1.068	-1.292	1.286	-1.077
	-1	-1	1	-1	-3.728	-1.071	-1	-1	-0.5	-1	1.3205	-1.339	-1.068	-1.292	1.286	-1.077

Inputs				t	z_{in1}	z_{in2}	z_1	z_2	y_{in}	y	w_{11}	w_{21}	b_1	w_{12}	w_{22}	b_2
	x_1	x_2	b													
Epoch 3	1	1	1	-1	-1.0865	-1.083	-1	-1	-0.5	-1	1.3205	-1.339	-1.068	-1.292	1.286	-1.077
	1	-1	1	1	1.5915	-3.655	1	-1	0.5	1	1.3205	-1.339	-1.068	-1.292	1.286	-1.077
	-1	1	1	1	-3.728	1.501	-1	1	0.5	1	1.3205	-1.339	-1.068	-1.292	1.286	-1.077
	-1	-1	1	-1	-1.0495	-1.071	-1	-1	-0.5	-1	1.3205	-1.339	-1.068	-1.292	1.286	-1.077

Delta Learning Rule

Architecture



- ❖ 3 The delta learning rule is only valid for continuous activation functions.
- ❖ It is a supervised training mode.
- ❖ The single weight w is updated using the following increment:

$$\Delta w_i = c(d_i - o_i)f'(net_i)x$$

- ❖ Where $f'(net_i)$ is the derivate of the activation function $f(net_i)$.

Application

Example

$w1 = [1 \ -1 \ 0 \ 0.5]$ $X1 = [1 \ -2 \ 0 \ -1]$ $x2 = [0 \ 1.5 \ -0.5 \ -1]$ $x3 = [-1 \ 1 \ 0.5 \ -1]$

$$\begin{bmatrix} x_1 = \begin{pmatrix} 1 \\ -2 \\ 0 \\ -1 \end{pmatrix} & x_2 = \begin{pmatrix} 0 \\ 1.5 \\ -0.5 \\ -1 \end{pmatrix} & x_3 = \begin{pmatrix} -1 \\ 1 \\ 0.5 \\ -1 \end{pmatrix} & w_1 = \begin{pmatrix} 1 \\ -1 \\ 0 \\ 0.5 \end{pmatrix} \end{bmatrix}$$

$c = 0.1$ and $\lambda = 1$. The desired responses for x_1 , x_2 , and x_3 are $d_1 = -1$, $d_2 = -1$, $d_3 = 1$. Use continues bipolar

activation function $f(\text{net}) = \frac{2}{1 + \exp(-\lambda \text{net})} - 1$

$$\begin{aligned} f'(\text{net}) &= \frac{(1 + \exp(-\lambda \text{net}))(0) - (2)(-\lambda \exp(-\lambda \text{net}))}{(1 + \exp(-\lambda \text{net}))^2} - 0 \\ &= \frac{2\lambda \exp(-\lambda \text{net})}{(1 + \exp(-\lambda \text{net}))^2} \\ &= \frac{\lambda}{2} \left[1 - \left(\frac{2}{1 + \exp(-\lambda \text{net})} - 1 \right)^2 \right] \\ &= \frac{\lambda}{2} (1 - O_i^2) \end{aligned}$$

For $\lambda = 1$

$$F'(\text{net}) = \frac{1}{2} (1 - O_i^2)$$

Step 1 :-

Input is x^1 , desired output is d^1 :

$$\text{Net}^1 = w^{1t} * X^1 = [1 \ -1 \ 0 \ 0.5] \begin{pmatrix} 1 \\ -2 \\ 0 \\ -1 \end{pmatrix} = 2.5$$

$$O^1 = f(\text{net}^1) = \frac{2}{1 + e^{-2.5}} - 1 = 0.848 \quad d^1 \neq O^1$$

We thus obtain updated weight vector

$$\begin{aligned} \Delta w^1 &= c (d^1 - O^1) * f'(\text{net}^1) * X^1 \\ &= 0.1 * (-1 - 0.848) (0.140) * \begin{pmatrix} 1 \\ -2 \\ 0 \\ -1 \end{pmatrix} = \begin{pmatrix} -0.0258 \\ 0.0517 \\ 0 \\ 0.0258 \end{pmatrix} \end{aligned}$$

$$\begin{aligned} W^{i+1} &= \Delta w^i + W^i \\ W^2 = \Delta w^1 + W^1 &= \begin{pmatrix} -0.0258 \\ 0.0517 \\ 0 \\ 0.0258 \end{pmatrix} + \begin{pmatrix} 1 \\ -1 \\ 0 \\ 0.5 \end{pmatrix} = \begin{pmatrix} 0.974 \\ -0.948 \\ 0 \\ 0.526 \end{pmatrix} \end{aligned}$$

Step 2 :-

Input is x_2 , desired output is d_2 :

$$F(\text{Net}^2) = w^{2t} * X^2 = \begin{pmatrix} 0.974 \\ -0.948 \\ 0 \\ 0.526 \end{pmatrix}^T * \begin{pmatrix} 0 \\ 1.5 \\ -0.5 \\ -1 \end{pmatrix} = -1.948$$

$$O^2 = f(\text{net}^2) = \frac{1}{1 + e^{-1.948}} = -0.750 \quad d_i \neq O_i$$

We thus obtain updated weight vector

$$f'(\text{net}^2) = \frac{1}{2} [1 - (O^2)^2] = \frac{1}{2} [1 - (-0.750)^2] = 0.218$$

$$\Delta w^2 = c (d^2 - O^2) * f'(\text{net}^2) * X^2$$

$$= 0.1 * (-1 + 0.750) * 0.218 * \begin{pmatrix} 0 \\ 1.5 \\ -0.5 \\ -1 \end{pmatrix} = \begin{pmatrix} 0 \\ -8.175 * 10^{-3} \\ 2.725 * 10^{-3} \\ 5.45 * 10^{-3} \end{pmatrix}$$

$$W^{i+1} = \Delta w^i + W^i$$

$$W^3 = \Delta w^2 + W^2 = \begin{pmatrix} 0 \\ -8.175 * 10^{-3} \\ 2.725 * 10^{-3} \\ 5.45 * 10^{-3} \end{pmatrix} + \begin{pmatrix} 0.974 \\ -0.948 \\ 0 \\ 0.526 \end{pmatrix} = \begin{pmatrix} 0.974 \\ -0.956 \\ 0.002 \\ 0.531 \end{pmatrix}$$

Step 3 :-

Input is x^3 , desired output is d^3 :

$$F(\text{Net}^3) = w^{3t} * X^3 = \begin{pmatrix} 0.974 \\ -0.956 \\ 0.002 \\ 0.531 \end{pmatrix}^T * \begin{pmatrix} -1 \\ 1 \\ 0.5 \\ -0.5 \end{pmatrix} = -2.46$$

$$O^3 = f(\text{net}^3) = \frac{2}{1 + e^{-2.46}} - 1 = -0.842 \quad d_i \neq O_i$$

We thus obtain updated weight vector

$$f'(\text{net}^3) = \frac{1}{2} [1 - (O^3)^2] = \frac{1}{2} [1 - (-0.842)^2] = 0.145$$

$$\begin{aligned} \Delta w^3 &= c (d^3 - O^3) * f'(\text{net}^3) * X^3 \\ &= 0.1 * (1 + 0.842) * 0.145 * \begin{pmatrix} -1 \\ 1 \\ 0.5 \\ -0.5 \end{pmatrix} = \begin{pmatrix} -0.026 \\ 0.026 \\ 0.0133 \\ -0.0133 \end{pmatrix} \end{aligned}$$

$$W^{t+1} = \Delta w^t + W^t$$

$$W^4 = \Delta w^3 + W^3 = \begin{pmatrix} -0.026 \\ 0.026 \\ 0.0133 \\ -0.0133 \end{pmatrix} + \begin{pmatrix} 0.974 \\ -0.956 \\ 0.002 \\ 0.531 \end{pmatrix} = \begin{pmatrix} 0.974 \\ -0.929 \\ 0.016 \\ 0.505 \end{pmatrix}$$